

Inżynieria danych

Wykład 7: indeksy (cz. I)

Katarzyna Grygiel

Semestr letni 2025/2026

Pliki uporządkowane to ciągle mało

Często zapytania dotyczą niewielkiej liczby rekordów z pliku:

```
SELECT * FROM pracownicy WHERE wydział = 'Fizyka';  
SELECT AVG(ocena) FROM oceny WHERE student_id = 107;
```

Pliki uporządkowane to ciągle mało

Często zapytania dotyczą niewielkiej liczby rekordów z pliku:

```
SELECT * FROM pracownicy WHERE wydział = 'Fizyka';  
SELECT AVG(ocena) FROM oceny WHERE student_id = 107;
```

W takiej sytuacji odczytywanie wszystkich krotek z tabel pracownicy i studenci byłoby nieefektywne.

Pliki uporządkowane to ciągle mało

Często zapytania dotyczą niewielkiej liczby rekordów z pliku:

```
SELECT * FROM pracownicy WHERE wydział = 'Fizyka';  
SELECT AVG(ocena) FROM oceny WHERE student_id = 107;
```

W takiej sytuacji odczytywanie wszystkich krotek z tabel pracownicy i studenci byłoby nieefektywne.

Wyszukanie ustalonej krotki w tabeli o długości N posortowanej względem interesującego nas atrybutu wymaga wykonania $\mathcal{O}(\log N)$ porównań i takiej samej liczbyostępów do dysku.

Pliki uporządkowane to ciągle mało

Często zapytania dotyczą niewielkiej liczby rekordów z pliku:

```
SELECT * FROM pracownicy WHERE wydział = 'Fizyka';  
SELECT AVG(ocena) FROM oceny WHERE student_id = 107;
```

W takiej sytuacji odczytywanie wszystkich krotek z tabel pracownicy i studenci byłoby nieefektywne.

Wyszukanie ustalonej krotki w tabeli o długości N posortowanej względem interesującego nas atrybutu wymaga wykonania $\mathcal{O}(\log N)$ porównań i takiej samej liczbyostępów do dysku.

Tabele nie są jednak posortowane i nawet przy wykorzystaniu plików uporządkowanych nie da się zagwarantować szybkiego przeszukiwania względem innego atrybutu niż atrybut porządkujący.

Indeksy

Indeksem nazywamy strukturę danych, w której przechowywane są tylko te atrybuty, względem których odbywa się sortowanie. Takie atrybuty nazywamy **kluczami wyszukiwania**. Nie muszą one stanowić żadnego z kluczy ani superkluczy relacji. Wraz z kluczem wyszukiwania przechowywany jest adres fizyczny odpowiedniej krotki.

Indeksy powinny być tak zaprojektowane, aby efektywne było wykonywanie nie tylko operacji wyszukiwania, ale i dodawania nowych oraz usuwania istniejących elementów.

SZBD a indeksy

System bazodanowy jest odpowiedzialny za utrzymywanie synchronizacji pomiędzy zawartością tabeli i indeksu.

SZBD a indeksy

System bazodanowy jest odpowiedzialny za utrzymywanie synchronizacji pomiędzy zawartością tabeli i indeksu.

Dodatkowo zadaniem SZBD jest określenie, który z istniejących indeksów powinien być wykorzystany przy wykonywaniu danego zapytania.

SZBD a indeksy

System bazodanowy jest odpowiedzialny za utrzymywanie synchronizacji pomiędzy zawartością tabeli i indeksu.

Dodatkowo zadaniem SZBD jest określenie, który z istniejących indeksów powinien być wykorzystany przy wykonywaniu danego zapytania.

Domyślnie większość systemów tworzy dla każdej tabeli indeksy z kluczem wyszukiwania odpowiadającym kluczowi głównemu oraz z kluczem dla każdego atrybutu unikalnego.

SZBD a indeksy

System bazodanowy jest odpowiedzialny za utrzymywanie synchronizacji pomiędzy zawartością tabeli i indeksu.

Dodatkowo zadaniem SZBD jest określenie, który z istniejących indeksów powinien być wykorzystany przy wykonywaniu danego zapytania.

Domyślnie większość systemów tworzy dla każdej tabeli indeksy z kluczem wyszukiwania odpowiadającym kluczowi głównemu oraz z kluczem dla każdego atrybutu unikalnego.

Im więcej indeksów, tym szybsze jest wyszukiwanie, ale równocześnie wzrastają koszty utrzymywania każdego z nich oraz rośnie ilość zajmowanego miejsca. W związku z tym nowe indeksy należy tworzyć rozważnie, biorąc pod uwagę dodatkowy czas potrzebny na ich aktualizowanie oraz dodatkową pamięć na ich przechowywanie.

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,
- każdy wierzchołek poza korzeniem zawiera co najmniej $M/2 - 1$ i co najwyżej $M - 1$ kluczy,

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,
- każdy wierzchołek poza korzeniem zawiera co najmniej $M/2 - 1$ i co najwyżej $M - 1$ kluczy,
- każdy węzeł wewnętrzny o k kluczach ma $k + 1$ potomków,

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,
- każdy wierzchołek poza korzeniem zawiera co najmniej $M/2 - 1$ i co najwyżej $M - 1$ kluczy,
- każdy węzeł wewnętrzny o k kluczach ma $k + 1$ potomków,
- klucze bazują na wartościach atrybutów, na podstawie których jest tworzony indeks,

B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,
- każdy wierzchołek poza korzeniem zawiera co najmniej $M/2 - 1$ i co najwyżej $M - 1$ kluczy,
- każdy węzeł wewnętrzny o k kluczach ma $k + 1$ potomków,
- klucze bazują na wartościach atrybutów, na podstawie których jest tworzony indeks,
- węzły wewnętrzne zawierają tablice z parami $\langle \text{node}^*, \text{key} \rangle$,

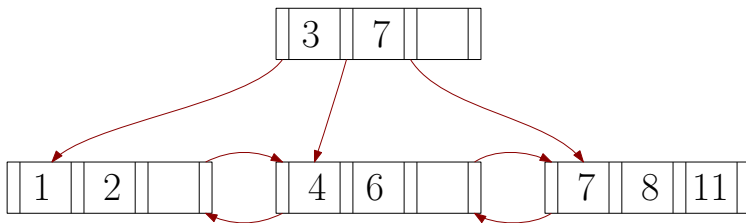
B+-drzewa

B+-drzewo to struktura danych, która przechowuje dane posortowane według pewnego klucza i umożliwia operacje przeszukiwania, dostępu sekwencyjnego, wstawiania i usuwania w czasie $\mathcal{O}(\log N)$.

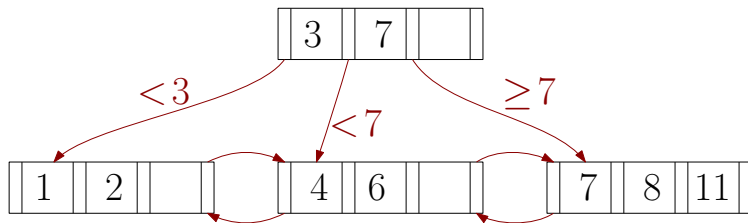
Charakterystyka B+-drzewa:

- drzewo ukorzenione, zbalansowane, M -arne,
- każdy wierzchołek poza korzeniem zawiera co najmniej $M/2 - 1$ i co najwyżej $M - 1$ kluczy,
- każdy węzeł wewnętrzny o k kluczach ma $k + 1$ potomków,
- klucze bazują na wartościach atrybutów, na podstawie których jest tworzony indeks,
- węzły wewnętrzne zawierają tablice z parami $\langle \text{node}^*, \text{key} \rangle$,
- liście zawierają tablice z parami $\langle \text{value}, \text{key} \rangle$.

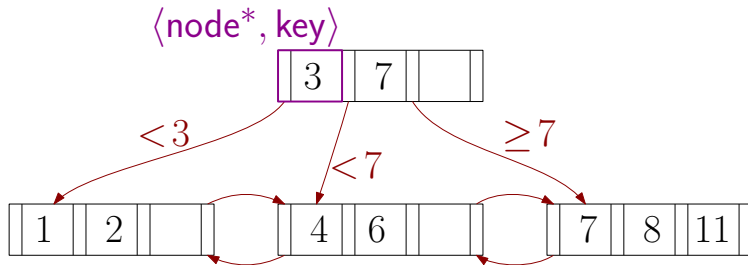
Przykład B+-drzewa



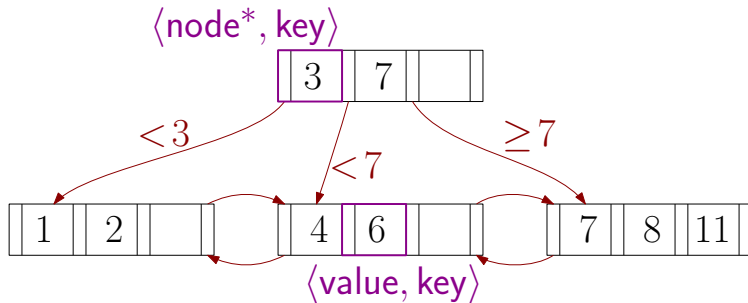
Przykład B+-drzewa



Przykład B+-drzewa



Przykład B+-drzewa



Liście w B+-drzewach

Wartościami w liściach mogą być całe krotki lub adresy bloków na dysku z odpowiednimi krotkami. To drugie rozwiązanie jest spotykane częściej.

Liście w B+-drzewach

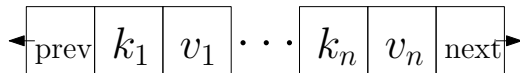
Wartościami w liściach mogą być całe krotki lub adresy bloków na dysku z odpowiednimi krotkami. To drugie rozwiązanie jest spotykane częściej.

Niezależnie od przechowywanych wartości, informacje w liściach mogą być zapisane na dwa sposoby.

Liście w B+-drzewach

Wartościami w liściach mogą być całe krotki lub adresy bloków na dysku z odpowiednimi krotkami. To drugie rozwiązanie jest spotykane częściej.

Niezależnie od przechowywanych wartości, informacje w liściach mogą być zapisane na dwa sposoby.



Liście w B+-drzewach

Wartościami w liściach mogą być całe krotki lub adresy bloków na dysku z odpowiednimi krotkami. To drugie rozwiązanie jest spotykane częściej.

Niezależnie od przechowywanych wartości, informacje w liściach mogą być zapisane na dwa sposoby.

poziom	liczba slotów	prev*	next*
--------	---------------	-------	-------

k_1	k_2	k_3	k_4	k_5	$\cdot \cdot \cdot$	k_n
-------	-------	-------	-------	-------	---------------------	-------

v_1	v_2	v_3	v_4	v_5	$\cdot \cdot \cdot$	v_n
-------	-------	-------	-------	-------	---------------------	-------

Przeszukiwanie w B+-drzewie

Przeszukiwanie rozpoczynamy od korzenia.

Każdy węzeł przeszukujemy przeglądając kolejne klucze. Ponieważ najczęściej tablice z kluczami są posortowane, to możemy przeszukiwać binarnie.

Przeszukiwanie w B+-drzewie

Przeszukiwanie rozpoczynamy od korzenia.

Każdy węzeł przeszukujemy przeglądając kolejne klucze. Ponieważ najczęściej tablice z kluczami są posortowane, to możemy przeszukiwać binarnie.

Gdy znajdziemy wartość większą lub równą poszukiwanej, to schodzimy do poddrzewa wyznaczonego przez znaleziony klucz.

Jeśli wszystkie klucze są większe od poszukiwanej wartości, to schodzimy do skrajnie lewego poddrzewa.

Przeszukiwanie w B+-drzewie

Przeszukiwanie rozpoczynamy od korzenia.

Każdy węzeł przeszukujemy przeglądając kolejne klucze. Ponieważ najczęściej tablice z kluczami są posortowane, to możemy przeszukiwać binarnie.

Gdy znajdziemy wartość większą lub równą poszukiwanej, to schodzimy do poddrzewa wyznaczonego przez znaleziony klucz.

Jeśli wszystkie klucze są większe od poszukiwanej wartości, to schodzimy do skrajnie lewego poddrzewa.

Czas przeszukiwania B+-drzewa o arności M i o N elementach wynosi $\mathcal{O}(\log_{M/2} N)$.

demo

Wstawianie w B+-drzewie

Dla wstawianego klucza znajdujemy odpowiedni liść.

Jeśli w liściu jest wolne miejsce, to wstawiamy nowy element zachowując porządek.

Wstawianie w B+-drzewie

Dla wstawianego klucza znajdujemy odpowiedni liść.

Jeśli w liściu jest wolne miejsce, to wstawiamy nowy element zachowując porządek.

Jeśli liść jest pełny (czyli zawiera już $M - 1$ kluczy), to dzielimy go na dwa liście i wstawiamy do każdego z nich po (prawie) tyle samo elementów, zachowując przy tym porządek. W rodzicu nowych liści dodajemy odpowiedni klucz, który będzie separatorem dla nowo powstałych liści. Jeśli również rodzic będzie przepełniony, postępujemy rekurencyjnie.

Wstawianie w B+-drzewie

Dla wstawianego klucza znajdujemy odpowiedni liść.

Jeśli w liściu jest wolne miejsce, to wstawiamy nowy element zachowując porządek.

Jeśli liść jest pełny (czyli zawiera już $M - 1$ kluczy), to dzielimy go na dwa liście i wstawiamy do każdego z nich po (prawie) tyle samo elementów, zachowując przy tym porządek. W rodzicu nowych liści dodajemy odpowiedni klucz, który będzie separatorem dla nowo powstałych liści. Jeśli również rodzic będzie przepełniony, postępujemy rekurencyjnie.

Czas wstawiania nowego elementu do B+-drzewa o arności M i o N wartościach wynosi $\mathcal{O}(M \log_{M/2} N)$.

demo

Usuwanie w B+-drzewach

Znajdujemy liść zawierający odpowiedni element i go usuwamy.

Jeśli liść zawiera nadal co najmniej $M/2 - 1$ elementów, to kończymy.

Usuwanie w B+-drzewach

Znajdujemy liść zawierający odpowiedni element i go usuwamy.

Jeśli liść zawiera nadal co najmniej $M/2 - 1$ elementów, to kończymy.

Jeśli jednak liść zawiera teraz za mało elementów (czyli mniej niż $M/2 - 1$), to sprawdzamy, czy możemy przenieść do niego skrajny element z sąsiadujących liści (o tym samym rodzicu). Gdy jest to możliwe, to uzupełniamy liść i uaktualniamy klucze w rodzicu. Gdy takie przeniesienie nie jest możliwe, to łączymy liść z jego sąsiadem. W tej sytuacji usuwamy klucz separujący łączone właśnie liście.

Usuwanie w B+-drzewach

Znajdujemy liść zawierający odpowiedni element i go usuwamy.

Jeśli liść zawiera nadal co najmniej $M/2 - 1$ elementów, to kończymy.

Jeśli jednak liść zawiera teraz za mało elementów (czyli mniej niż $M/2 - 1$), to sprawdzamy, czy możemy przenieść do niego skrajny element z sąsiadujących liści (o tym samym rodzicu). Gdy jest to możliwe, to uzupełniamy liść i uaktualniamy klucze w rodzicu. Gdy takie przeniesienie nie jest możliwe, to łączymy liść z jego sąsiadem. W tej sytuacji usuwamy klucz separujący łączone właśnie liście.

Jeśli po tej operacji węzeł wewnętrzny ma za mało elementów, postępujemy rekurencyjnie.

Czas usuwania elementu z B+-drzewa o arności M i o N wartościach wynosi $\mathcal{O}(M \log_{M/2} N)$.

demo

Rzeczywiste B+-drzewa

Według badań eksperymentalnych typowy stopień zapęśnienia drzewa dla prawdziwej bazy z prawdziwymi danymi to ok. 67% przy typowej arności drzewa równej 200 (średnio 133 klucze na węzeł).

Rzeczywiste B+-drzewa

Według badań eksperymentalnych typowy stopień wypełnienia drzewa dla prawdziwej bazy z prawdziwymi danymi to ok. 67% przy typowej arności drzewa równej 200 (średnio 133 klucze na węzeł).

Więcej „średnich” statystyk:

- łączna pojemność węzłów:
 - na poziomie 3: $133^3 = 2\,406\,104$,
 - na poziomie 4: $133^4 = 312\,900\,721$,
- liczba stron na każdym z poziomów:
 - poziom 1: 8 kB (1 strona),
 - poziom 2: 1 MB (133 strony),
 - poziom 3: 133 MB (17 689 stron).

Rzeczywiste B+-drzewa

Według badań eksperymentalnych typowy stopień zapełnienia drzewa dla prawdziwej bazy z prawdziwymi danymi to ok. 67% przy typowej arności drzewa równej 200 (średnio 133 klucze na węzeł).

Więcej „średnich” statystyk:

- łączna pojemność węzłów:
 - na poziomie 3: $133^3 = 2\,406\,104$,
 - na poziomie 4: $133^4 = 312\,900\,721$,
- liczba stron na każdym z poziomów:
 - poziom 1: 8 kB (1 strona),
 - poziom 2: 1 MB (133 strony),
 - poziom 3: 133 MB (17 689 stron).

Niektóre systemy bazodanowe nie od razu łączą węzły, gdy znajdzie się w nich zbyt mało kluczy. Jest to spowodowane tym, że operacja łączenia jest stosunkowo kosztowna. Co więcej, czasami opłaca się dopuścić do deficytowych węzłów i co jakiś czas zbudować całe drzewo od nowa.

Rozmiary węzłów

Ogólna zasada: im wolniejsza pamięć, w której przechowywane jest drzewo, tym większy jest optymalny rozmiar węzła w B+-drzewie.

Rozmiary węzłów

Ogólna zasada: im wolniejsza pamięć, w której przechowywane jest drzewo, tym większy jest optymalny rozmiar węzła w B+-drzewie.

Jeśli przyjmiemy, że węzeł mieści się w jednym bloku pamięci, to otrzymujemy następujące przybliżone rozmiary węzłów:

- dysk HDD – ok. 1 MB,
- dysk SSD – ok. 10 kB,
- RAM – ok. 512 B.

Rozmiary węzłów

Ogólna zasada: im wolniejsza pamięć, w której przechowywane jest drzewo, tym większy jest optymalny rozmiar węzła w B+-drzewie.

Jeśli przyjmiemy, że węzeł mieści się w jednym bloku pamięci, to otrzymujemy następujące przybliżone rozmiary węzłów:

- dysk HDD – ok. 1 MB,
- dysk SSD – ok. 10 kB,
- RAM – ok. 512 B.

Rozmiar może również zależeć od charakteru zapytań, które będą dominować w bazie.

W przypadku częstych zapytań o zbiór krotek, dla których wartości atrybutu, według którego utworzony jest indeks, należą do pewnego zakresu, optymalne będą duże węzły, z których informacje będą odczytywane sekwencyjnie. Z drugiej strony jeśli dominują zapytania o pojedyncze krotki, to mniejszy rozmiar węzła będzie lepszy.

Przykład

Rozważmy sytuację, w której znamy następujące wartości:

- rozmiar pliku: $r = 100000$ rekordów,
- rozmiar strony: $B = 8$ kB,
- rozmiar rekordu: $R = 100$ B,
- rozmiar klucza: $K = 10$ B,
- rozmiar wartości: $V = 20$ B.

Przykład

Rozważmy sytuację, w której znamy następujące wartości:

- rozmiar pliku: $r = 100000$ rekordów,
- rozmiar strony: $B = 8$ kB,
- rozmiar rekordu: $R = 100$ B,
- rozmiar klucza: $K = 10$ B,
- rozmiar wartości: $V = 20$ B.

Maksymalna liczba rekordów na jednej stronie wynosi

$$rb = \left\lfloor \frac{B}{R} \right\rfloor = \left\lfloor \frac{8192}{100} \right\rfloor = 81,$$

Przykład

Rozważmy sytuację, w której znamy następujące wartości:

- rozmiar pliku: $r = 100000$ rekordów,
- rozmiar strony: $B = 8$ kB,
- rozmiar rekordu: $R = 100$ B,
- rozmiar klucza: $K = 10$ B,
- rozmiar wartości: $V = 20$ B.

Maksymalna liczba rekordów na jednej stronie wynosi

$$rb = \left\lfloor \frac{B}{R} \right\rfloor = \left\lfloor \frac{8192}{100} \right\rfloor = 81,$$

natomiast minimalna liczba stron z danymi to

$$\left\lceil \frac{r}{rb} \right\rceil = \left\lceil \frac{100000}{81} \right\rceil = 1235.$$

Przykład

Jaka może być minimalna wysokość B^+ -drzewa, w którym uda nam się przechować adresy wszystkich rekordów?

Przykład

Jaka może być minimalna wysokość $B+$ -drzewa, w którym uda nam się przechować adresy wszystkich rekordów?

Arność drzewa M powinna spełniać następującą nierówność:

$$M \cdot V + (M - 1) \cdot K \leq B,$$

co nam daje $M \leq 273$.

Przykład

Jaka może być minimalna wysokość $B+$ -drzewa, w którym uda nam się przechować adresy wszystkich rekordów?

Arność drzewa M powinna spełniać następującą nierówność:

$$M \cdot V + (M - 1) \cdot K \leq B,$$

co nam daje $M \leq 273$. Zatem wysokość $B+$ -drzewa wynosi

$$\lceil \log_M r \rceil = \lceil \log_{273} 100000 \rceil = 3.$$

Przykład

Jaka może być minimalna wysokość $B+$ -drzewa, w którym uda nam się przechować adresy wszystkich rekordów?

Arność drzewa M powinna spełniać następującą nierówność:

$$M \cdot V + (M - 1) \cdot K \leq B,$$

co nam daje $M \leq 273$. Zatem wysokość $B+$ -drzewa wynosi

$$\lceil \log_M r \rceil = \lceil \log_{273} 100000 \rceil = 3.$$

Jeśli skorzystalibyśmy z wyszukiwania binarnego, to liczba odczytów z dysku wynosiłaby $\lceil \log_2 100000 \rceil = 17$. Otrzymujemy zatem ponad pięciokrotne przyspieszenie.

Co zrobić dla kluczy zmiennego rozmiaru?

Gdy rozmiar klucza wyszukiwania jest zmienny, to możemy:

- przechowywać klucze jako wskaźniki do wartości atrybutu (metoda już niestosowana),

Co zrobić dla kluczy zmiennego rozmiaru?

Gdy rozmiar klucza wyszukiwania jest zmienny, to możemy:

- przechowywać klucze jako wskaźniki do wartości atrybutu (metoda już niestosowana),
- dopuścić różne rozmiary węzłów, zależne od rozmiaru klucza (również nieużywana),

Co zrobić dla kluczy zmiennego rozmiaru?

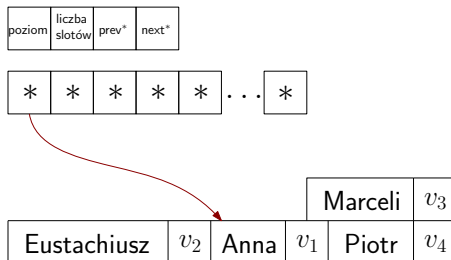
Gdy rozmiar klucza wyszukiwania jest zmienny, to możemy:

- przechowywać klucze jako wskaźniki do wartości atrybutu (metoda już niestosowana),
- dopuścić różne rozmiary węzłów, zależne od rozmiaru klucza (również nieużywana),
- dopełnić każdy klucz do maksymalnego rozmiaru dla typu danego klucza,

Co zrobić dla kluczy zmiennego rozmiaru?

Gdy rozmiar klucza wyszukiwania jest zmienny, to możemy:

- przechowywać klucze jako wskaźniki do wartości atrybutu (metoda już niestosowana),
- dopuścić różne rozmiary węzłów, zależne od rozmiaru klucza (również nieużywana),
- dopełnić każdy klucz do maksymalnego rozmiaru dla typu danego klucza,
- przechowywać klucze podobnie jak w przypadku strony ze slotami:



B+-drzewa są fajne

Kilka powodów, dla których stosuje się B+-drzewa:

- pozwalają na bardzo małą liczbę odczytów z dysku,

B+-drzewa są fajne

Kilka powodów, dla których stosuje się B+-drzewa:

- pozwalają na bardzo małą liczbę odczytów z dysku,
- działają dobrze na każdym poziomie hierarchii pamięci,

B+-drzewa są fajne

Kilka powodów, dla których stosuje się B+-drzewa:

- pozwalają na bardzo małą liczbę odczytów z dysku,
- działają dobrze na każdym poziomie hierarchii pamięci,
- zapewniają kompromis między dostępem swobodnym i sekwencyjnym,

B+-drzewa są fajne

Kilka powodów, dla których stosuje się B+-drzewa:

- pozwalają na bardzo małą liczbę odczytów z dysku,
- działają dobrze na każdym poziomie hierarchii pamięci,
- zapewniają kompromis między dostępem swobodnym i sekwencyjnym,
- radzą sobie z różnymi typami danych,

B+-drzewa są fajne

Kilka powodów, dla których stosuje się B+-drzewa:

- pozwalają na bardzo małą liczbę odczytów z dysku,
- działają dobrze na każdym poziomie hierarchii pamięci,
- zapewniają kompromis między dostępem swobodnym i sekwencyjnym,
- radzą sobie z różnymi typami danych,
- sprawdzają się dobrze nie tylko dla warunków równościowych, ale i dla zakresów.